Model Evaluation

Model Evaluation and Selection

- Positive tuples- tuples of the main class of interest
- Negative tuples- tuples of the other class

- True Positives- positive tuples correctly labeled
- True Negatives- negative tuples correctly labeled
- False Positives- negative tuples incorrectly labeled
- False Negatives- positive tuples incorrectly labeled

Evaluation Metrics- Confusion Matrix

A confusion matrix is a table that is often used to describe the performance of a classification model (or "classifier") on a set of test data for which the true values are known.

- It allows the visualization of the performance of an algorithm.
- We compare each class with every other class and see how many samples are misclassified.
- We actually come across several key metrics that are very important in the field of machine learning and Deep Learning.

Evaluation Metrics- Confusion Matrix

Let's consider a binary classification case where the output is either 0 or 1:

1. **True positives:** These are the samples for which we predicted 1 as the output and the ground truth is 1 too.
2. **True negatives:** These are the samples for which we predicted 0 as the output and the ground truth is 0 too.
3. **False positives:** These are the samples for which we predicted 1 as the output but the ground truth is 0. This is also known as a Type I error.
4. **False negatives:** These are the samples for which we predicted 0 as the output but the ground truth is 1. This is also known as a Type II error.

Evaluation Metrics- Confusion Matrix

True Positive(TP):

- You projected positive and its turn out to be true. For example, you had predicted that France would win the world cup, and it won.

True Negative(TN):

- When you predicted negative, and it's true. You had predicted that England would not win and it lost.

False Positive(FP):

- Your prediction is positive, and it is false.
- You had predicted that England would win, but it lost.

False Negative(FN):

- Your prediction is negative, and result it is also false.
- You had predicted that France would not win, but it won.

Evaluation Metrics- Confusion Matrix

		Predicted Class: Positive	Predicted Class: Negative	
Actual Class	**Positive**	True Positive (TP)	False Negative (FN) **Type II Error**	**Sensitivity** $\frac{TP}{(TP+FN)}$
	Negative	False Positive (FP) **Type I Error**	True Negative (TN)	**Specificity** $\frac{TN}{(TN+FP)}$
		Precision $\frac{TP}{(TP+FP)}$	**Negative Predictive Value** $\frac{TN}{(TN+FN)}$	**Accuracy** $\frac{TP+TN}{(TP+TN+FP+FN)}$

Confusion Matrix - example

- Suppose we have a binary classification problem to predict whether an email is **Spam** or **Not Spam**. Here, we have four outcomes in our confusion matrix:
- **True Positive (TP)**: Predicted Spam, and it is actually Spam.
- **True Negative (TN)**: Predicted Not Spam, and it is actually Not Spam.
- **False Positive (FP)**: Predicted Spam, but it is actually Not Spam (also known as a Type I error).
- **False Negative (FN)**: Predicted Not Spam, but it is actually Spam (also known as a Type II error).

Confusion Matrix - example

Let's say we test our model with 10 emails, and we have the following results:

Actual	Predicted
Spam	Spam
Not Spam	Spam
Spam	Spam
Not Spam	Not Spam
Not Spam	Spam
Spam	Not Spam
Not Spam	Not Spam
Spam	Spam
Not Spam	Not Spam
Spam	Not Spam

Calculate TP, TN, FP, FN

Based on the results above, we categorize each outcome:

1. **True Positives (TP)**: Predicted Spam and actually Spam → 3
2. **True Negatives (TN)**: Predicted Not Spam and actually Not Spam → 3
3. **False Positives (FP)**: Predicted Spam but actually Not Spam → 2
4. **False Negatives (FN)**: Predicted Not Spam but actually Spam → 2

Confusion Matrix - example

Create the Confusion Matrix

Using the counts from above, we can create the confusion matrix:

	Predicted: Spam	Predicted: Not Spam
Actual Spam	TP = 3	FN = 2
Actual Not Spam	FP = 2	TN = 3

Confusion Matrix:

$$\begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$$

Confusion Matrix - example

Actual class\ Predicted class	buy_computer = yes	buy_computer = no	Total
buy_computer = yes	**6954**	**46**	7000
buy_computer = no	**412**	**2588**	3000
Total	7366	2634	10000

Evaluation Metrics - Accuracy, Error Rate

A\P	C	¬C	
C	**TP**	**FN**	**P**
¬C	**FP**	**TN**	**N**
	P'	**N'**	**All**

- **Classifier Accuracy,** or recognition rate: percentage of test set tuples that are correctly classified

  Accuracy = (TP + TN)/All

- **Error rate:** *1 – accuracy*, or

  Error rate = (FP + FN)/All

Evaluation Metrics: Sensitivity, Specificity

A\P	C	¬C	
C	**TP**	**FN**	**P**
¬C	**FP**	**TN**	**N**
	P’	**N’**	**All**

- **Sensitivity**: True Positive recognition rate (TPR)
  - **Sensitivity = TP/P= TP/(TP+FN)**
- **Specificity**: True Negative recognition rate (TNR)
  - **Specificity = TN/N = TN/(TN+FP)**

Evaluation Metrics: Precision, Recall, and F-measures

- **Precision**: exactness – what % of tuples that the classifier labeled as positive are actually positive

$$precision = \frac{TP}{TP + FP}$$

- **Recall Or Sensitivity (**completeness) – what % of positive tuples did the classifier label as positive?

$$recall = \frac{TP}{TP + FN}$$

- Perfect score is 1.0
- Inverse relationship between precision & recall

Evaluation Metrics: Precision, Recall, and F-measures

- ***F* measure (*F*$_1$ or *F*-score)**: harmonic mean of precision and recall,

$$F = \frac{2 \times precision \times recall}{precision + recall}$$

Actual Class\Predicted class	cancer = yes	cancer = no	Total	Recognition(%)
cancer = yes	90	210	300	30.00 (*sensitivity)*
cancer = no	140	9560	9700	98.56 (*specificity)*
Total	230	9770	10000	96.40 (*accuracy*)

- *Precision* = 90/230 = 39.13% *Recall* = 90/300 = 30.00%

Other Performance Metrics

Using the confusion matrix, we can calculate various metrics to evaluate the model's performance:

1. **Accuracy**: Measures the overall correctness of the model.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{3+3}{3+3+2+2} = \frac{6}{10} = 0.6\,(60\%)$$

2. **Precision**: Measures the accuracy of positive predictions.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{3}{3+2} = \frac{3}{5} = 0.6\,(60\%)$$

3. **Recall** (Sensitivity): Measures the ability to find all positive cases.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{3}{3+2} = \frac{3}{5} = 0.6\,(60\%)$$

4. **F1 Score**: The harmonic mean of Precision and Recall.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{0.6 \times 0.6}{0.6 + 0.6} = 0.6\,(60\%)$$

Why do we need so many evaluation metrics?

A model can make different kinds of errors, and not all errors are equally important. That is why one metric alone cannot fully explain model performance.

<u>Key points</u>

- ❑ Accuracy shows overall correctness
- ❑ Precision shows how reliable positive predictions are
- ❑ Recall shows how many actual positives are found
- ❑ Specificity shows how well negatives are identified
- ❑ F1-score gives a balance between precision and recall

Different metrics answer different questions about the model.

Accuracy is useful, but not always enough

Accuracy works well when the dataset is **balanced**, but it can be misleading when the dataset is **imbalanced**.

Balanced data: A dataset is called **balanced** when the classes are present in nearly equal numbers.

Example:

- ✓ 50 spam emails
- ✓ 50 non-spam emails

In this case, accuracy can be a meaningful metric because both classes are represented fairly.

Imbalanced data: A dataset is called **imbalanced** when one class has many more samples than the other.

Example:

- ✓ 950 non-spam emails
- ✓ 50 spam emails

In this case, a model may get high accuracy simply by predicting the majority class most of the time.

Why accuracy alone can fail in imbalanced data

Example

Suppose we have **1000 emails**:

- ✓ 950 are Not Spam
- ✓ 50 are Spam

Now imagine a model that predicts **all emails as Not Spam**.

Result

- ✓ Correct = 950
- ✓ Wrong = 50

Accuracy

- ✓ Accuracy = 950 / 1000 = **95%**

But the real problem -

- ✓ Spam detected = **0**
- ✓ Recall for spam = **0%**

Conclusion: Even though the accuracy is very high, the model is actually useless for spam detection.

Why do we use precision, recall, specificity, and F1-score?

Precision

Question it answers: Out of all predicted positive cases, how many are truly positive?
Use when: False positives are costly.
Example: In spam detection, low precision means many normal emails are wrongly marked as spam.

Recall

Question it answers: Out of all actual positive cases, how many did the model correctly find?
Use when: False negatives are costly.
Example: In disease detection, low recall means many sick patients are missed.

Specificity

Question it answers: Out of all actual negative cases, how many did the model correctly identify as negative?
Use when: Correctly identifying negatives is important.
Example: In medical tests, high specificity means healthy people are less likely to be wrongly diagnosed as sick.

F1-score

Question it answers: How well does the model balance precision and recall?
Use when: You need a single score for imbalanced classification.
Example: If both false positives and false negatives matter, F1-score is often more informative than accuracy.

When should we focus on which metric?

If dataset is balanced

- ✓ Accuracy can be used
- ✓ But other metrics still help for deeper analysis

If dataset is imbalanced

- ✓ Do not rely on accuracy alone
- ✓ Focus more on:
  - Precision
  - Recall
  - F1-score
  - Specificity

<u>Real-life examples</u>

- ✓ **Disease detection** → Recall is very important
- ✓ **Spam filtering** → Precision is important
- ✓ **Fraud detection** → Recall and F1-score are important
- ✓ **General balanced classification** → Accuracy may be acceptable

<u>Conclusion</u>: **Accuracy tells only the overall success rate, but other metrics explain the quality of predictions and the types of mistakes made by the model. Therefore, proper model evaluation requires multiple metrics, especially in imbalanced datasets.**

The End